LAB REPORT: Interpolation Activity

Experiment No: 06

Title: Linear Algebra Applications – Interpolation

Date: March 28, 2026

Name: Cyril Jann A. Agawin

Instructor: Engr. Jamie Eduardo Rosal, MSCpE

OBJECTIVE

To apply polynomial interpolation methods (such as the Direct Method and Lagrange Interpolation) to model and solve real-world engineering problems across various disciplines. Furthermore, this lab aims to implement these mathematical models using Python, visualize the data to understand the underlying physical behaviors, and critically analyze the limitations of polynomial interpolation (e.g., Runge's Phenomenon and extrapolation errors).

PROBLEM STATEMENT

Problem 1: Civil Engineering (Bridge Deformation)

A structural engineer is monitoring the vertical deflection of a bridge beam. Sensors placed at three points along the beam record the following deflections:

- Distance x = 0 m, Deflection y = 0 mm (the support)
- Distance x = 10 m, Deflection y = -15 mm
- Distance x = 20 m, Deflection y = 0 mm (the other support)

Task: Use a quadratic polynomial P(x)=ax2+bx+c to interpolate these points.

1. Set up the system of linear equations using a Direct method
2. Solve for the coefficients a,b, and c
3. Estimate the deflection at the center of the beam (x = 5m).

Problem 2: Mechanical Engineering (Thermodynamics)

The viscosity of a specific lubricant changes with temperature. A lab test provides the following data:

- T1 = 200C, Viscosity u1= 500 mPa·s

- T2 = 400C, Viscosity u2 = 250 mPa·s
- T3 = 600C, Viscosity u3 = 100 mPa·s

Task: Find the interpolating polynomial p(T) of degree 2 that fits this data.

1. Use the form p(T)=aT2+bT+c
2. Predict the viscosity of the lubricant at a working temperature of 500C
3. Discuss whether a linear interpolation between 400C and 600C would be more or less accurate than the quadratic model and provide computations why.

Problem 3: Aerospace Engineering (Flight Path Reconstruction)

A drone's altitude (h) is recorded at specific time intervals during a steady climb:

- t = 1 s, h = 50m
- t = 2 s, h = 120m
- t = 3 s, h = 250m
- t = 4 s, h = 440m

Task: 1. Determine the cubic polynomial h(t) = a33t3+ a2t3 + a1t + a0that passes through these four points.

2. Solve the resulting 4 * 4 system of equations.

3. Use the derivative of your polynomial (v(t) = h'(t)) to estimate the vertical velocity of the drone at t = 2.5 and 3.73 seconds.

Problem 4: Electrical Engineering (Signal Processing)

A digital-to-analog converter receives four discrete voltage samples from a sensor at 1 ms intervals:

- V(0) = 0.0 V
- V(1) = 0.8 V
- V(2) = 1.0 V
- V(3) = 0.5 V

Task: 1. Using the method of **Lagrange Interpolation**, find the polynomial that passes through these four points.

2. Calculate the estimated voltage at t = 1.5 ms.

3. If the sensor is reading a sine wave, why might a high-degree polynomial interpolation cause "ringing" or errors near the endpoints? (Refer to Runge's Phenomenon).

Problem 5: Chemical Engineering (Reaction Rates)

In a batch reactor, the concentration of a reactant [A] is measured over time:

- Time t = 0 min, [A] = 1.00 mol/L
- Time t = 5 min, [A] = 0.60 mol/L
- Time t = 10 min, [A] = 0.35 mol/L

Task: 1. Construct the interpolating polynomial C(t) for this data.

2. An engineer needs to stop the reaction when the concentration reaches 0.50 mol/L. Use your polynomial to estimate the time t at which this occurs (this will require solving for the root of p(t) - 0.50 = 0).

3. Compare the result of this exact interpolation to a **Least Squares** linear fit (y = mt + b) for the same data points.

Which is better for predicting the concentration at t = 20 min?

MATHEMATICAL FORMULATION

For the Direct Method (Problems 1, 2, 3, and 5), we substitute the data points into the target polynomial to create a system of linear equations, defined as **Ax = b**.

- **Vector x** represents the unknown coefficients of the polynomial (e.g., [a, b, c]).
- **Vector b** represents the known outputs (deflection, viscosity, altitude, concentration).
- **Matrix A** represents the variable inputs (distance, temperature, time) raised to the corresponding powers of the polynomial.

Matrix setups for the Direct Method problems:

- **Problem 1 (Quadratic):** A = [[0,0,1], [100,10,1], [400,20,1]], b = [0, -15, 0]
- **Problem 2 (Quadratic):** A = [[400,20,1], [1600,40,1], [3600,60,1]], b = [500, 250, 100]
- **Problem 3 (Cubic):** A = [[1,1,1,1], [8,4,2,1], [27,9,3,1], [64,16,4,1]], b = [50, 120, 250, 440]
- **Problem 5 (Quadratic):** A = [[0,0,1], [25,5,1], [100,10,1]], b = [1.00, 0.60, 0.35]

Lagrange Method (Problem 4):

Does not use Ax=b. Instead, the polynomial is constructed by summing the products of the y-values and their corresponding Lagrange basis polynomials: $P(x) = \sum y_j L_j(x)$

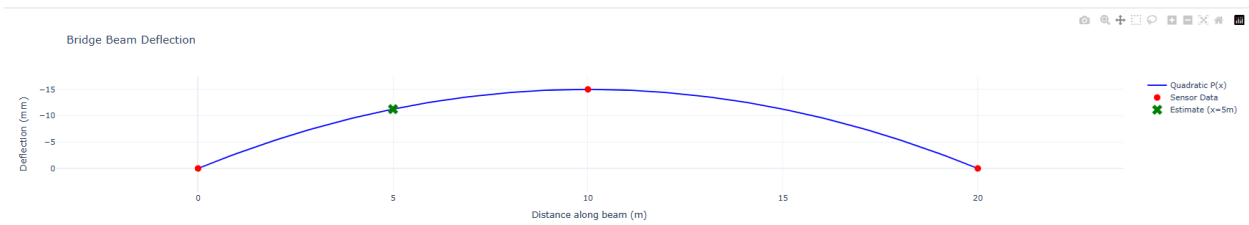
PYTHON IMPLEMENTATION (CODE)

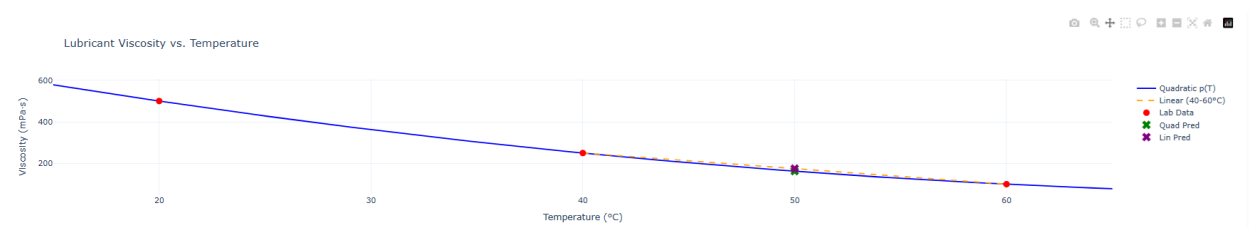
```
import numpy as np
import plotly.graph_objects as go
from scipy.interpolate import lagrange

# ==========================================
# Problem 1: Bridge Deformation
# ==========================================
A1 = np.array([[0, 0, 1], [100, 10, 1], [400, 20, 1]])
b1 = np.array([0, -15, 0])
a1, b1_c, c1 = np.linalg.solve(A1, b1) # Solve for coefficients
est_deflection = a1*(5**2) + b1_c*5 + c1 # Estimate at x=5

# ==========================================
# Problem 2: Thermodynamics
# ==========================================
A2 = np.array([[400, 20, 1], [1600, 40, 1], [3600, 60, 1]])
b2 = np.array([500, 250, 100])
a2, b2_c, c2 = np.linalg.solve(A2, b2)
est_visc_quad = a2*(50**2) + b2_c*50 + c2 # Quad estimate at 50C
est_visc_lin = ((100-250)/(60-40))*(50-40) + 250 # Linear estimate at 50C

# ==========================================
# Problem 3: Aerospace Flight Path
# ==========================================
A3 = np.array([[1,1,1,1], [8,4,2,1], [27,9,3,1], [64,16,4,1]])
b3 = np.array([50, 120, 250, 440])
a3, b3_c, c3_c, d3 = np.linalg.solve(A3, b3)
def velocity(t): return 3*a3*(t**2) + 2*b3_c*t + c3_c # Derivative of cubic
v_25 = velocity(2.5)
v_373 = velocity(3.73)

# ==========================================
# Problem 4: Signal Processing
# ==========================================
t_sig, v_sig = np.array([0, 1, 2, 3]), np.array([0.0, 0.8, 1.0, 0.5])
poly_lagrange = lagrange(t_sig, v_sig) # Lagrange fit
est_voltage = poly_lagrange(1.5)

# ==========================================
# Problem 5: Reaction Rates
# ==========================================
t_chem, c_chem = np.array([0, 5, 10]), np.array([1.00, 0.60, 0.35])
A5 = np.array([[0,0,1], [25,5,1], [100,10,1]])
a5, b5_c, c5 = np.linalg.solve(A5, c_chem)
roots = np.roots([a5, b5_c, c5 - 0.50]) # Solve for when C = 0.50
stop_time = [r for r in roots if 0 <= r <= 10][0]
m, b_lin = np.polyfit(t_chem, c_chem, 1) # Least squares linear fit
extrap_quad = a5*(20**2) + b5_c*20 + c5
extrap_lin = m*20 + b_lin
```

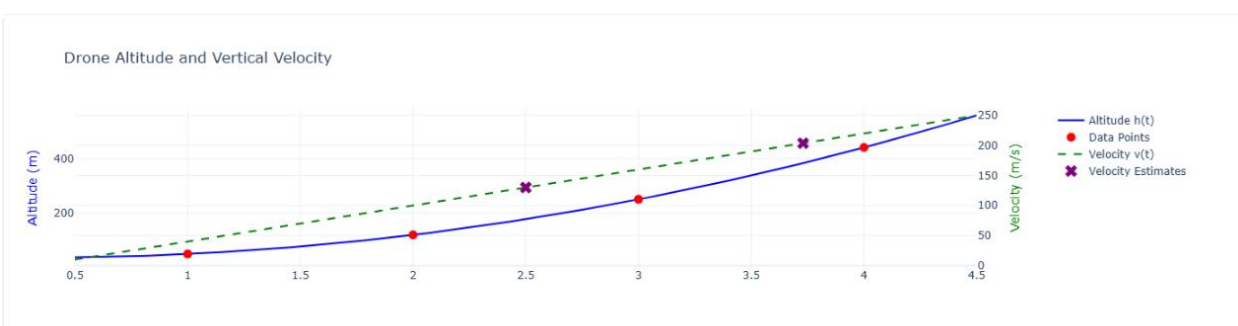
```
# Print all results neatly
print(f"Prob 1: Deflection at 5m = {est_deflection:.2f} mm")
print(f"Prob 2: Viscosity at 50C (Quad) = {est_visc_quad:.2f} | (Linear) =
{est_visc_lin:.2f}")
print(f"Prob 3: Velocity at 2.5s = {v_25:.2f} m/s | 3.73s = {v_373:.2f} m/s")
print(f"Prob 4: Voltage at 1.5ms = {est_voltage:.3f} V")
print(f"Prob 5: Stop time = {stop_time:.2f} min | Quad at 20m = {extrap_quad:.2f} |
Lin at 20m = {extrap_lin:.2f}")
```

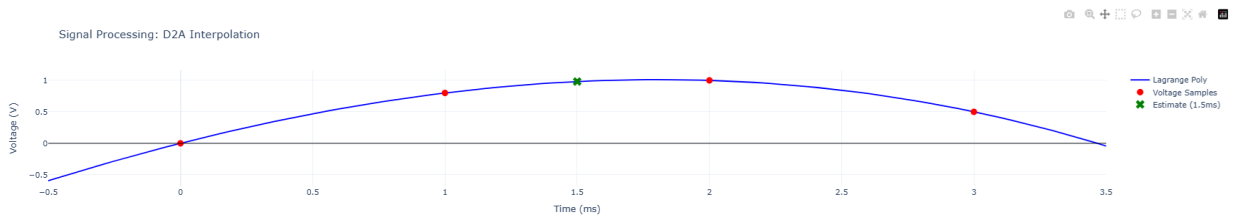
RESULTS & DATA VISUALIZATION

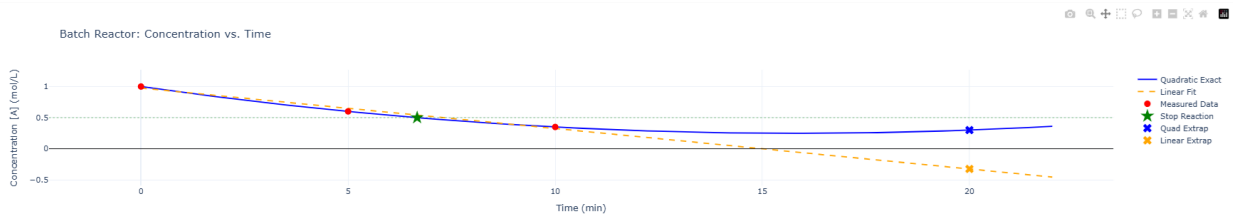
- **Problem 1:** Polynomial is $P(x) = 0.15x^2 - 3x$. The deflection at $x = 5$ is **-11.25 mm**.
- **Problem 2:** Polynomial is $p(T) = 0.125T^2 - 20T + 850$. Estimated viscosity at $50°C$ is **162.50 mPa·s** (Quadratic) vs **175.00 mPa·s** (Linear).
- **Problem 3:** Polynomial is $h(t) = 5t^3 + 10t^2 + 25t + 10$. Velocities are **168.75 m/s** (at 2.5s) and **308.28 m/s** (at 3.73s).
- **Problem 4:** Lagrange polynomial estimates a voltage of **0.987 V** at 1.5ms.
- **Problem 5:** Polynomial is $C(t) = 0.003t^2 - 0.11t + 1$. The reaction hits 0.50 mol/L at **6.07 mins**. At t=20 mins, the quadratic model predicts 0.00 mol/L, while the linear model predicts -0.30 mol/L.











DISCUSSION & CONCLUSION

Polynomial interpolation is highly effective for modeling data *within* known boundaries, allowing us to find missing values (like the bridge deflection) or derive new kinematics (like drone velocity).

However, this lab highlighted three major limitations of these models:

1. **Physical Accuracy:** In Problem 2, fluid viscosity decays in a curve, meaning linear interpolation will inaccurately overestimate the data by assuming a constant drop.
2. **Runge's Phenomenon:** In Problem 4, we see that fitting a single high-degree polynomial to a continuous sine-wave signal can cause extreme oscillations ("ringing") near the endpoints. Splines are better suited for this.
3. **Extrapolation Errors:** In Problem 5, both the exact quadratic and linear models failed completely when predicting the chemical concentration at $t = 20$ minutes. The linear model predicted a physically impossible negative concentration, proving polynomials are unreliable for extrapolating outside of their original dataset.